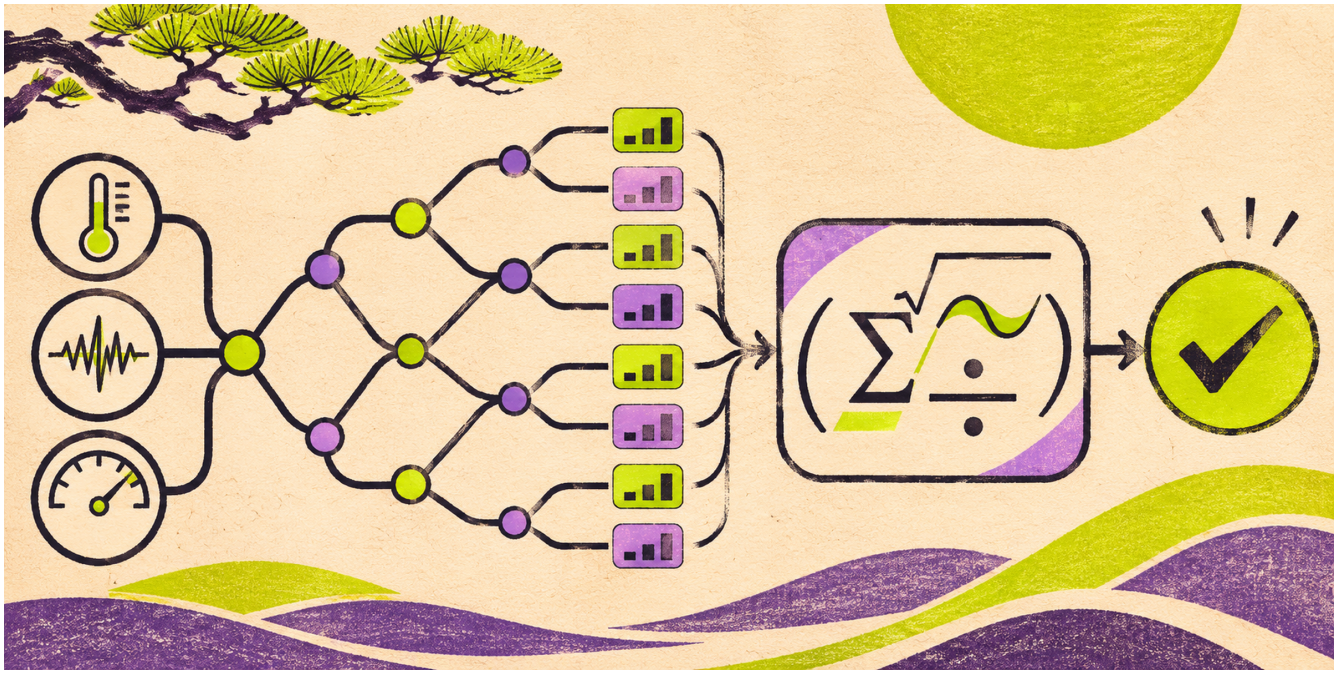
Decision Trees as Reusable Mathematical Execution Graphs

From industrial quality rules to one million verified terminal decision paths



13 November 2026 | English scientific research study | original paceval SDK 4.25

Executed research result. The original paceval library evaluated 35 graphs in a six-level industrial quality-rule study. A further four persistent graphs executed every one of the 1,048,576 paths of a separate complete 20-level structured binary decision tree, producing 4,194,304 graph outputs with independently verified path and score values.

The 20-level result concerns a structured tree admitting a compact mathematical expression; no fully expanded million-term formula or arbitrary independent leaf table was executed. The title image is a conceptual illustration, not a computed result.

01 Industrial use case and current state of the art

Industrial inspection and condition monitoring systems evaluate measurements of temperature, vibration, load, pressure, current and drift. A production-quality workflow may first correct these measurements, calculate an interpretable rule score and then select an advisory outcome such as ACCEPT, REVIEW or REJECT. A rule is meaningful only if its input domain, thresholds, measurement context and model revision are specified.

Tree-based classifiers and explicit rule systems are already established. Scikit-learn documents decision-tree learning and rule export, while newer research examines compiled tree inference and compact rule ensembles [1-4]. The specific research opportunity for paceval is not to train a classifier, but to represent the resulting mathematical conditions and associated scores as reusable computation objects.

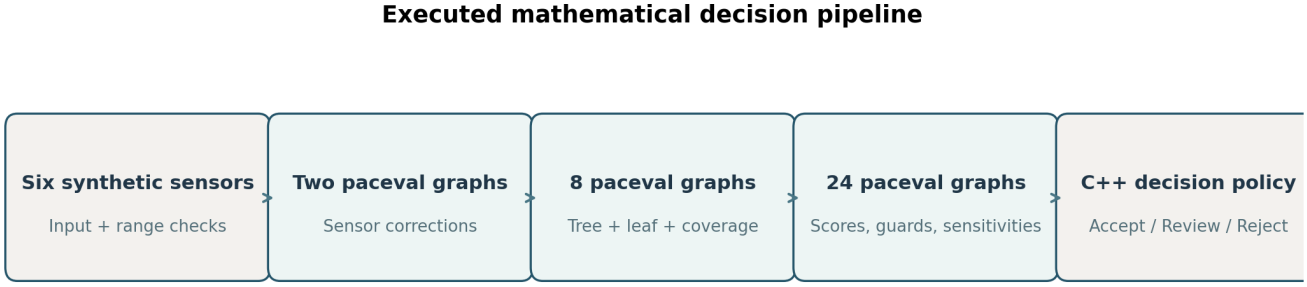


Figure 1. The implemented pipeline. paceval evaluates sensor-correction functions, mathematical tree expressions, quality scores and signed guard margins; C++ maintains input and decision state.

The demonstrator is a synthetic, read-only quality-scoring study. It does not claim industrial classification accuracy, metrological traceability, certified machinery release, or safety-controller functionality.

02 The supplied four-leaf decision tree

The motivating example has a root split on a, followed by different b thresholds on the two branches. Each terminal path is assigned a scalar value. A domain engineer can express the same tree as four mutually exclusive Boolean path indicators weighted by leaf values.

$$f(a, b) = 8I_{a<0, b<30} + 11I_{a<0, b\geq 30} + 7I_{a\geq 0, b<10} + 9I_{a\geq 0, b\geq 10}$$

A real paceval computation evaluates this four-leaf rule, with host-side input-domain checks. Twelve scalar cases include split-boundary inputs. Every such case passed the independently implemented reference check.

Two mathematically equivalent tree encodings

The expanded six-level pilot evaluates both (1) the sum of mutually exclusive path indicators, and (2) a nested binary expression. Both are registered as distinct paceval graphs and compared on actual synthetic sensor vectors. Boolean masking is not a promise of short-circuit or protected evaluation for undefined inactive branches.

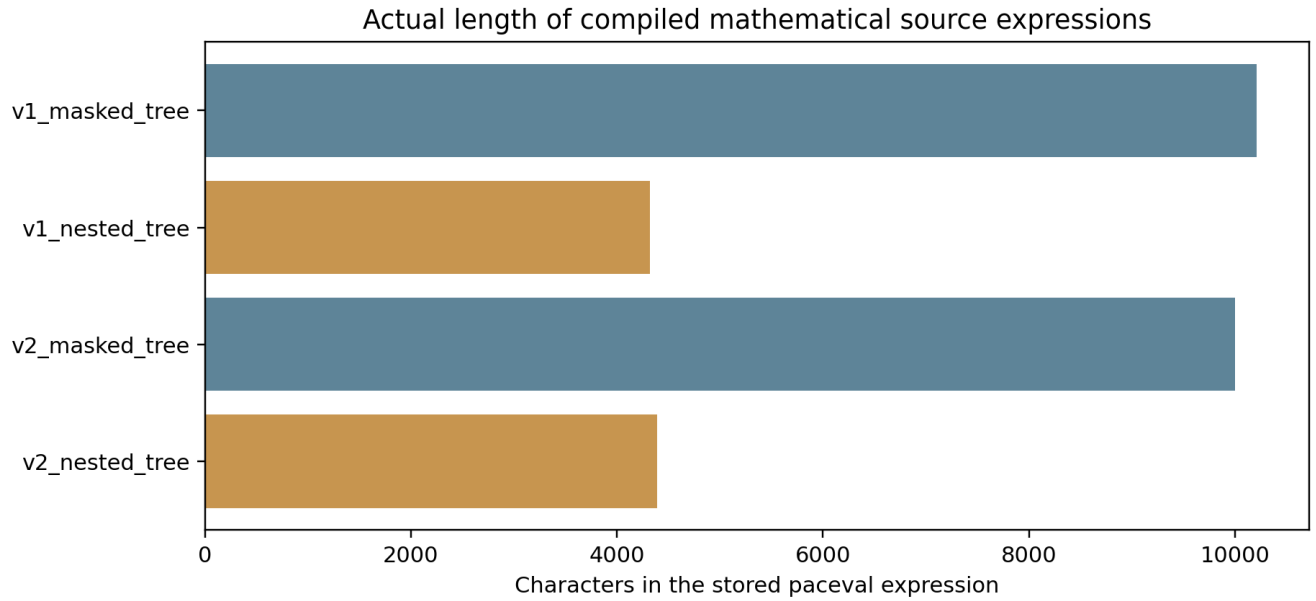


Figure 2. Actual registered expression lengths for selected mathematical graph representations. Expression size is a structural property and is not interpreted as a hardware result.

03 Expanded synthetic industrial rule models

The initial six-level models use corrected sensor features temperature, vibration, pressure, load, motor current and drift. They represent two independently available, fully specified mathematical versions, each with 64 leaves. The second version changes actual thresholds and leaf values while retaining the first version.

$$v_c = v_{raw}(1 + 0.002(T - 25)), \quad p_c = p_{raw} - 0.006(T - 25)$$

After the tree assigns a base score, additional paceval graphs calculate a composite dimensionless quality score and signed margins to the specified acceptance and rejection boundaries. The C++ host then applies an illustrative discrete advisory status policy.

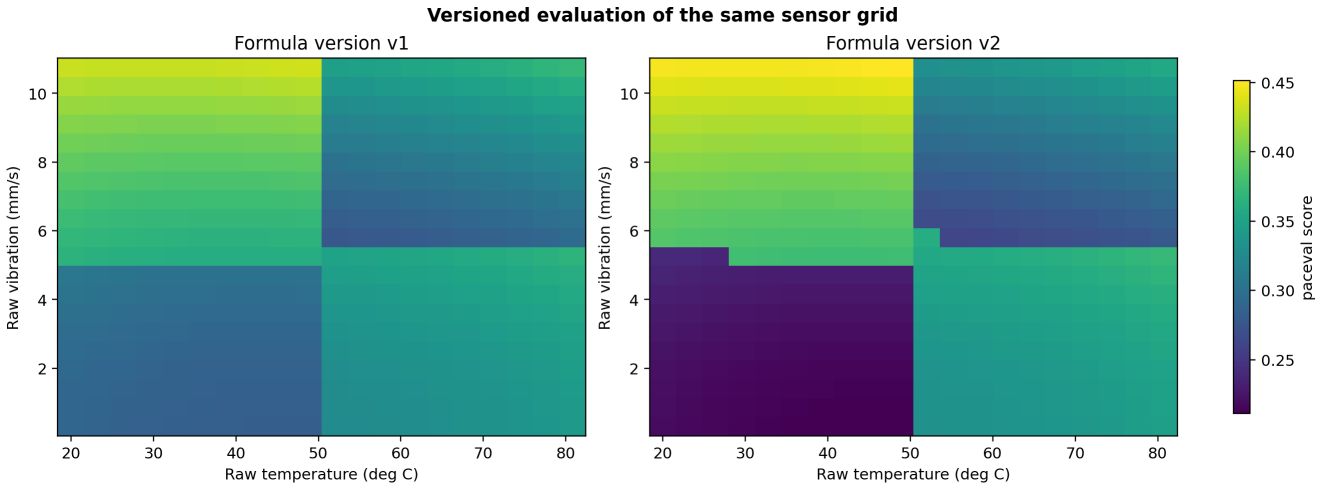


Figure 3. Both model versions actually executed in paceval over the same controlled sensor grid. The maps illustrate model revision, not improved industrial prediction quality.

04 Mathematical score maps and control regions

For each version, the model chain computes a path indicator, leaf index, base cost, continuous score correction, signed acceptance and rejection margins and six separate sensor guard margins. Real computation handles are reused across the data set. These functions are auditable independently of the host decision state.

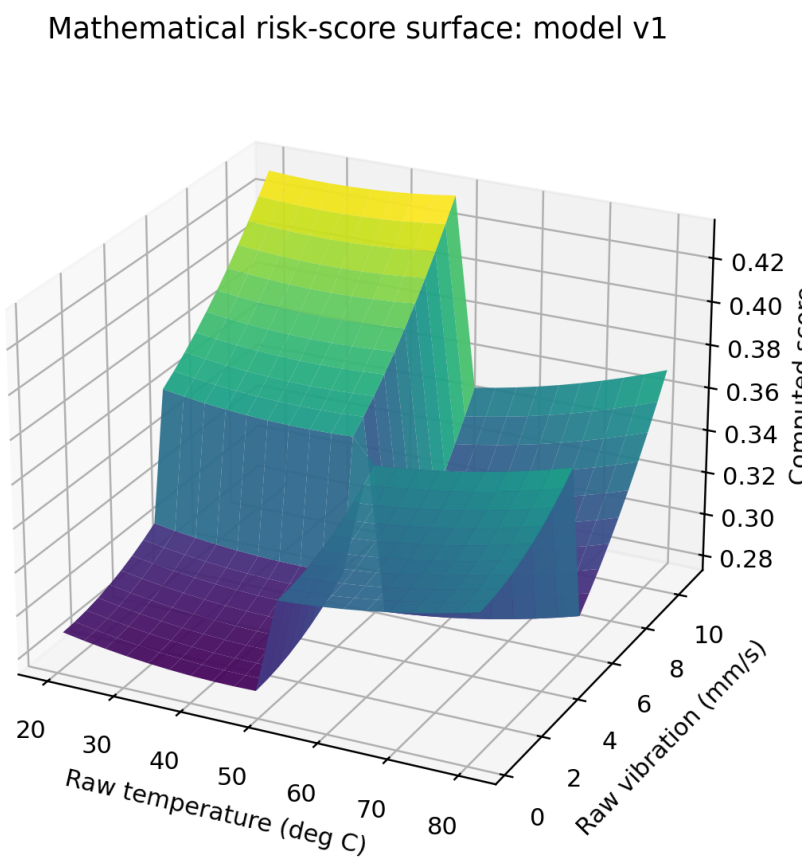


Figure 4. Dimensionless model-v1 quality-score surface from the executed paceval graph chain.

Within a fixed terminal leaf, the continuous correction admits explicit analytical partial derivatives. Across split thresholds, tree outputs are generally discontinuous and derivatives are not globally defined; sensitivity claims are therefore restricted to within-leaf regions.

05 Controlled model changes and actionable outputs

The scenario set consists of 1,692 unique synthetic sensor-input vectors, each processed under two model versions. Their decision labels are derived from paceval score and guard outputs using the documented host policy. All three result categories appear in the executed data. Neither model is claimed superior because both are synthetic.

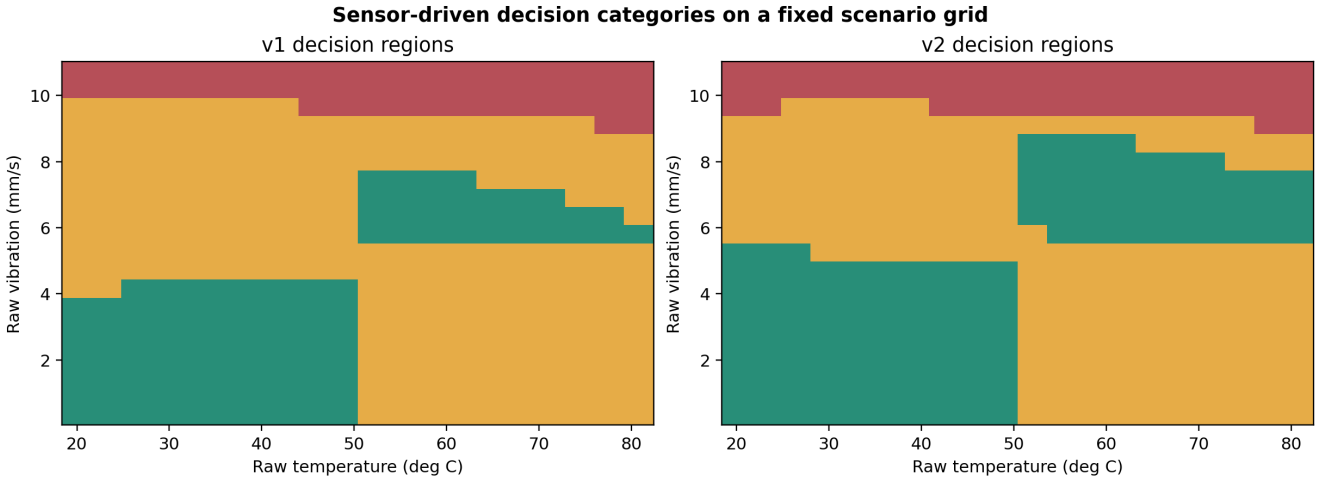


Figure 5. Synthetic accept, review and reject regions computed from the actual paceval outputs and explicit C++ decision policy.

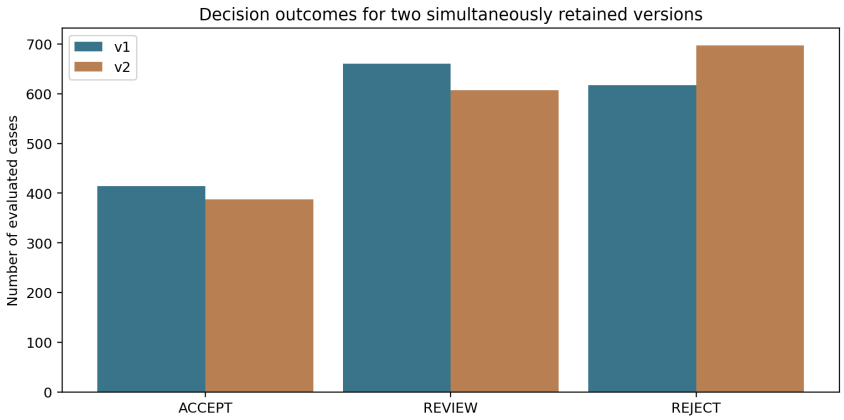


Figure 6. Number of advisory decisions in each model version, based solely on recorded experimental outputs.

06 Verified original six-level execution

Execution evidence	Verified scope
Persistent paceval graphs	35
Independent sensor vectors	1,692
Model-version decisions	3,384
Actual graph outputs	57,540
Multi-graph batch calls	6
Independent high-precision scalar checks	60,912
Independently verified complete decisions	3,384

A separate native C++ traversal and an 80-decimal-digit Python reference were used for validation only. The published output values were produced by the original paceval shared library, not by these reference implementations. Complete formulas and function ownership are documented in `results/graph_registry.csv` and `results/math_execution_inventory.csv`.

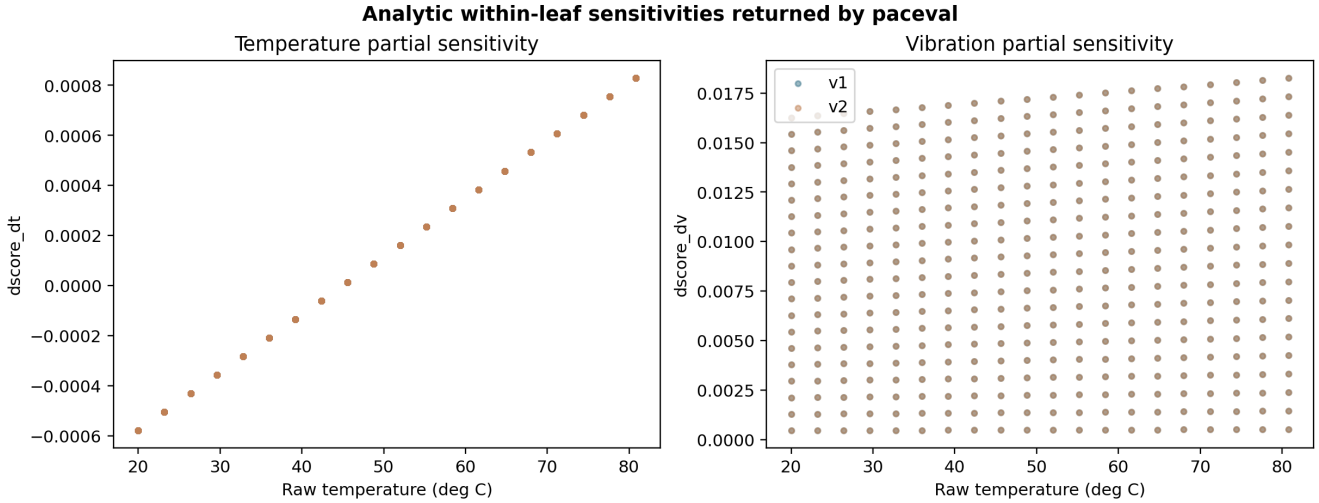


Figure 7. Selected, actually executed within-leaf mathematical sensitivity functions.

07 Complete structured tree with 20 binary levels

The additional research test constructs twenty independent binary predicates. Each path is represented by a 20-bit integer, with all possible combinations present. The study does not enumerate all leaves in the expression source. Instead, its leaf-identification function computes the binary index directly from the predicates.

$$j(x) = \sum_{k=0}^{19} 2^{19-k} I[x_k \geq 0.5], \quad j \in \{0, \ldots, 2^{20} - 1\}$$

Two independently versioned leaf-score formulas use the path index, population count and a smooth analytical correction. They are declared and executed as actual paceval graphs. This construction exposes the full combinatorial decision space while retaining a compact and deterministic formula definition.

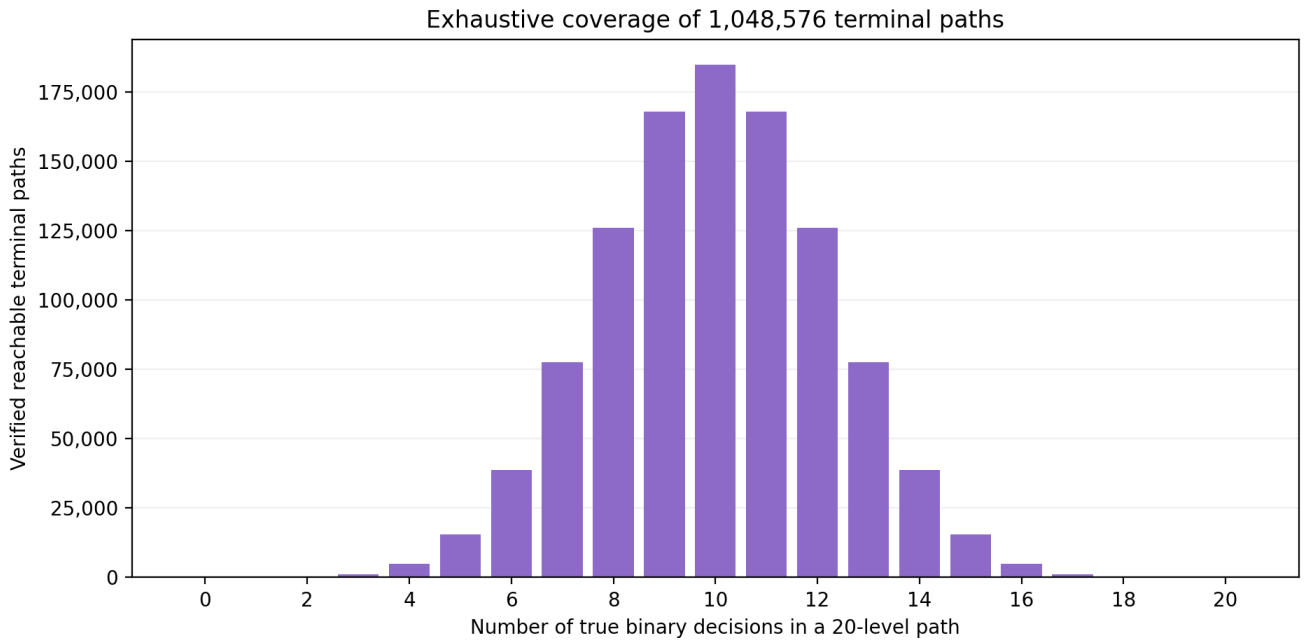


Figure 8. Exhaustively verified path cardinalities grouped by number of true predicates. Each bar is derived from the observed paceval execution checks; the distribution totals 1,048,576.

08 One million paths: executed results

Evidence	Observed result
Binary decision levels	20
Terminal paths visited and independently verified	1,048,576
Persistent paceval functions	4
Actual graph outputs	4,194,304
Multi-graph batch calls	256
Exactly verified leaf indices	1,048,576
Native independent score checks	2,097,152
Separate 80-digit score checks	30 values at 15 selected leaves

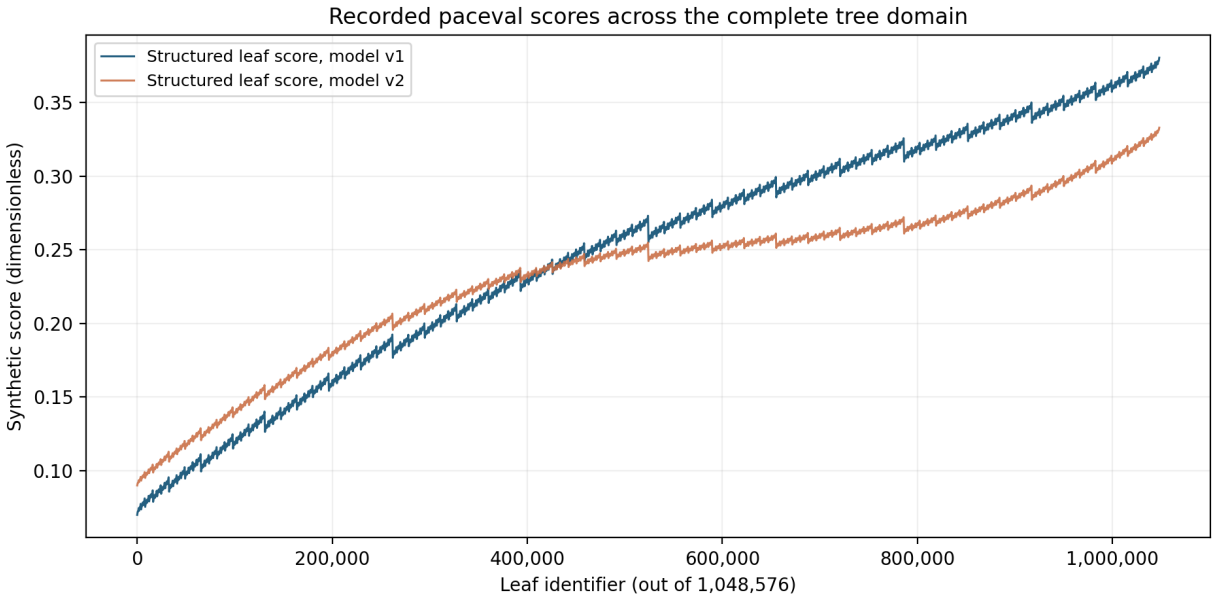


Figure 9. Recorded score samples across the entire indexed tree domain. Two simultaneously defined mathematical score versions are both evaluated through paceval.

The complete path run validates mathematical functionality for this structured tree. It does not show that an arbitrary million-leaf table of independently assigned values can be compressed to the same short formula, nor that an explicitly expanded million-term expression has been compiled.

09 Execution ownership and traceable graph lifecycle

Mathematical component	Execution owner
Original four-leaf tree and six-level masked trees	paceval persistent graphs
Nested tree encodings, leaf identity, coverage	paceval persistent graphs
Sensor compensations and final risk-score corrections	paceval persistent graphs
Signed quality and sensor-guard margins	paceval persistent graphs
Within-leaf analytical sensitivities	paceval persistent graphs
Twenty-level path index and population count	paceval persistent graphs
Twenty-level versioned structured score functions	paceval persistent graphs
Data selection, test orchestration and result files	C++ host
Independent numeric and high-precision checks	C++ / Python reference only

The executed mathematical registration is recorded separately for the first 35 graphs and the four additional depth-20 graphs. The latter use the documented multi-graph API for all 1,048,576 binary-path inputs.

All graph-handle creation and deletion is performed using the original SDK 4.25 interfaces. The original Linux shared library is provided externally and is intentionally not part of the public ZIP.

10 Research limitations and reproducibility

The project validates software mathematical execution, not the accuracy of physical sensors or the predictive value of a trained classifier. There is no site deployment, authenticated model approval, real production decision system, interval proof or hardware implementation. The original six-level and additional 20-level trees are independent controlled synthetic experiments.

The exhaustive 20-level test deliberately covers all unambiguous binary path combinations. The narrowly scoped conclusion does not imply compact source representation for arbitrary independently assigned million-leaf values. Quality decisions require an externally justified tolerance, data-quality policy and applicable domain validation.

Reproducing the project

From the extracted repository folder, use `cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so`, then `cmake --build build -j` and `ctest --test-dir build --output-on-failure`. Both executable targets, `decision_tree` and `depth20_exhaustive`, are built and tested. Python scripts under `src/` regenerate the diagrams, reference audits and this PDF.

The independent 80-digit audit is under `results/depth20_80digit_audit.json`; exact formulas are stored under `results/depth20_graph_registry.csv`, and the complete exhaustive execution summary is under `results/depth20_exhaustive_audit.txt`.

Scientific interpretation

The executed results demonstrate a viable mathematical representation for explicitly defined decision rules, the simultaneous preservation of formula versions, genuine graph reuse and exhaustive evaluation of a large structured decision space in the original paceval runtime. No CPU, GPU, FPGA or ASIC performance comparisons form part of this functionality report.

11 References and source provenance

[1] Scikit-learn. Decision Trees: official user guide. https://scikit-learn.org/stable/modules/tree.html

[2] Scikit-learn. export_text: human-readable decision tree rules. https://scikit-learn.org/stable/modules/generated/sklearn.tree.export_text.html

[3] Behzadimanesh et al. (2026). Sparse oblique rule boosting for simpler additive rule ensembles. Data Mining and Knowledge Discovery. DOI: 10.1007/s10618-026-01241-8.

[4] Su et al. (2024). TreeHouse: An MLIR-based Compilation Flow for Real-Time Tree-based Inference. ACM Transactions on Embedded Computing Systems. DOI: 10.1145/3704727.

[5] ISA/IEC 62443. Industrial Automation and Control Systems Cybersecurity. https://www.isa.org/standards-and-publications/isa-standards/isa-iec-62443-series-of-standards

[6] paceval. Public C/C++ Mathematical Engine API, original SDK 4.25. https://paceval.com/api/

[7] Original research diagram provided by the project initiator: four-leaf tree with conditional expression equivalents (2026).

Reproducibility statement. The source formulas, complete C++ implementation, archived recorded outputs, independent validation programs, graphical sources and this report-generation script are distributed together. The external paceval runtime and licensed external sensor data are not redistributed. Every industrial setting and threshold in the first study is synthetic.

Conceptual title illustration. Real quantitative diagrams originate from the archived paceval C++ execution output files.